

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM301 Алгоритмын шинжилгээ ба зохиомж
2025-2026 оны хичээлийн жилийн намрын улирал

Бие даалтын тайлан №1

Хичээл заасан багш:

Д.Батмөнх

Багийн дугаар:

№11

Гүйцэтгэсэн оюутан:

Ц.Төгөлдөр (B232270050)

Г.Нармандах (B232270057)

Э.Амарболд (B232270048)

Б.Тэмүүлэн (B232270042)

Улаанбаатар хот
2025 он

Агуулга

1	БИЕ ДААЛТЫН ЗОРИЛГО	1
1.1	Программын зорилго	1
2	АЛГОРИТМЫН ХЭРЭГЛЭЭ	2
2.1	BFS (Breadth-First Search) алгоритм	2
2.2	DFS (Depth-First Search) алгоритм	3
2.3	Dijkstra алгоритм	4
3	ПРОГРАММЫН АЖИЛЛАГАА БА ХЭРЭГЖҮҮЛЭЛТ	6
3.1	Төслийн ерөнхий бүтэц	6
3.2	Өгөгдөл боловсруулах шат	6
3.3	Backend хэсгийн ажиллах зарчим	7
3.4	Frontend хэсгийн хэрэгжүүлэлт	7
4	Класс диаграмм	9
5	ХЭРЭГЛЭГЧИЙН ИНТЕРФЕЙС	11
5.1	Ерөнхий зохион байгуулалт	11
5.2	Газрын зураг дээрх харилцан үйлдэл	11
5.3	Алгоритм сонголт ба зам тооцоолол	11
5.4	Маршрутын дүрслэл ба үр дүнгийн мэдээлэл	12
5.5	Алдаа болон зааварчилгын мессежүүд	12
5.6	Хэрэглэгчийн үйлдлийн ерөнхий дараалал	12
5.7	Алгоритмуудын дүрслэл	13
6	ТЕСТ БА БАТАЛГААЖУУЛАЛТ	15
6.1	Зорилго	15
6.2	Инвариант дээр суурилсан баталгаажуулалт	15
6.3	Интеграцын болон гар тестүүд	16
6.4	Тестийн үр дүнгийн дүгнэлт	16
6.5	Туршилтын үр дүнгийн хүснэгт (Python үр дүн)	17
6.6	Туршилтын үр дүнгийн хүснэгт (Java үр дүн)	17
6.7	Алгоритмуудын дундаж гүйцэтгэлийн харьцуулалт (Python vs Java)	17
7	ДҮГНЭЛТ	18
7.1	Ерөнхий дүгнэлт	18
7.2	Алгоритмуудын харьцуулалтын дүгнэлт	18
7.3	Программын хэрэгжилтийн ололт амжилт	18
7.4	Хязгаарлалт ба цаашдын сайжруулалт	18
7.5	Хувийн суралцах үр дүн	19

1 БИЕ ДААЛТЫН ЗОРИЛГО

1.1 Программын зорилго

Энэхүү бие даалтын зорилго нь графын хайлтын **BFS**, **DFS**, **Dijkstra** зэрэг алгоритмуудын онолын үндэс, ажиллах зарчмыг судалж [1, 4, 5, 6], тэдгээрийг Улаанбаатар хотын газрын зураг дээр автомашинаар явах **богино замыг тооцоолох** веб суурьтай системд хэрэгжүүлэхэд оршино.

Эдгээр алгоритмуудыг ашигласнаар графын өгөгдөлд дүн шинжилгээ хийх, замын холболтын бүтэцтэй танилцах, хамгийн оновчтой маршрутыг тодорхойлох зэрэг бодит хэрэглээтэй асуудлуудыг шийдвэрлэх боломж бүрдэнэ. Төслийн хүрээнд дараах үндсэн зорилтуудыг дэвшүүлж, хэрэгжүүлэв:

- Улаанбаатар хотын OpenStreetMap-ийн shapefile өгөгдлөөс замын сүлжээг графын *node*, *edge* бүтэц болгон хөрвүүлэх [3, 2].
- Хэрэглэгчийн сонгосон хоёр цэгийн хооронд автомашинаар зорчиж болох боломжит замыг граф дээр загварчилж, хайлт хийх.
- **BFS**, **DFS**, **Dijkstra** алгоритм бүрийг ашиглан:
 - хамгийн цөөн алхамтай (ирмэгийн тоо бага) замыг,
 - жингийн хувьд хамгийн бага нийт зардалтай (замын урт, хугацаа багатай) замыг,
 - алгоритмуудын гаргаж буй замын ялгаа, онцлогийг тодорхойлон харьцуулах.
- Алгоритмуудын гүйцэтгэлийг (тооцооллын хугацаа, замын урт, алхмын тоо зэрэг) харьцуулж, аль алгоритм ямар нөхцөлд илүү тохиромжтойг дүгнэх.

Ингэснээр онолын төвшинд үзсэн графын алгоритмуудыг бодит газрын зургийн өгөгдөл дээр хэрэгжүүлж, практик ур чадвар эзэмшихийн зэрэгцээ алгоритм бүрийн давуу ба сул талыг туршилтаар баталгаажуулах нь энэ бие даалтын үндсэн ач холбогдол юм.

2 АЛГОРИТМЫН ХЭРЭГЛЭЭ

Энэхүү бие даалтад графын гурван төрлийн хайлтын алгоритмыг ашигласан. Үүнд: **BFS (Breadth-First Search)**, **DFS (Depth-First Search)**, **Dijkstra** алгоритмууд бөгөөд тэдгээрийг Улаанбаатар хотын замын граф дээр хэрэгжүүлж, хооронд нь үр дүн ба гүйцэтгэлийн хувьд харьцуулсан.

2.1 BFS (Breadth-First Search) алгоритм

BFS алгоритм нь графын оройнуудыг **төвшин дарааллаар** шалгадаг өргөн чиглэлтэй хайлтын арга юм. Эхлэх оройгоос хамгийн ойрхон хөршүүдийг эхэлж шалгаад, дараа нь дараагийн төвшний хөршүүд уруу шилжинэ. Ирмэгийн жин тооцохгүй тул алхмын тоогоор хамгийн богино замыг олдог.

Манай төсөлд BFS алгоритмыг:

- Хоёр цэгийн хооронд **хамгийн цөөн ирмэгтэй (алхам багатай)** маршрутыг тодорхойлоход,
- Алгоритм жинг тооцдоггүй үед ямар төрлийн зам гарч ирдгийг жишээ болгон үзүүлэхэд ашигласан.

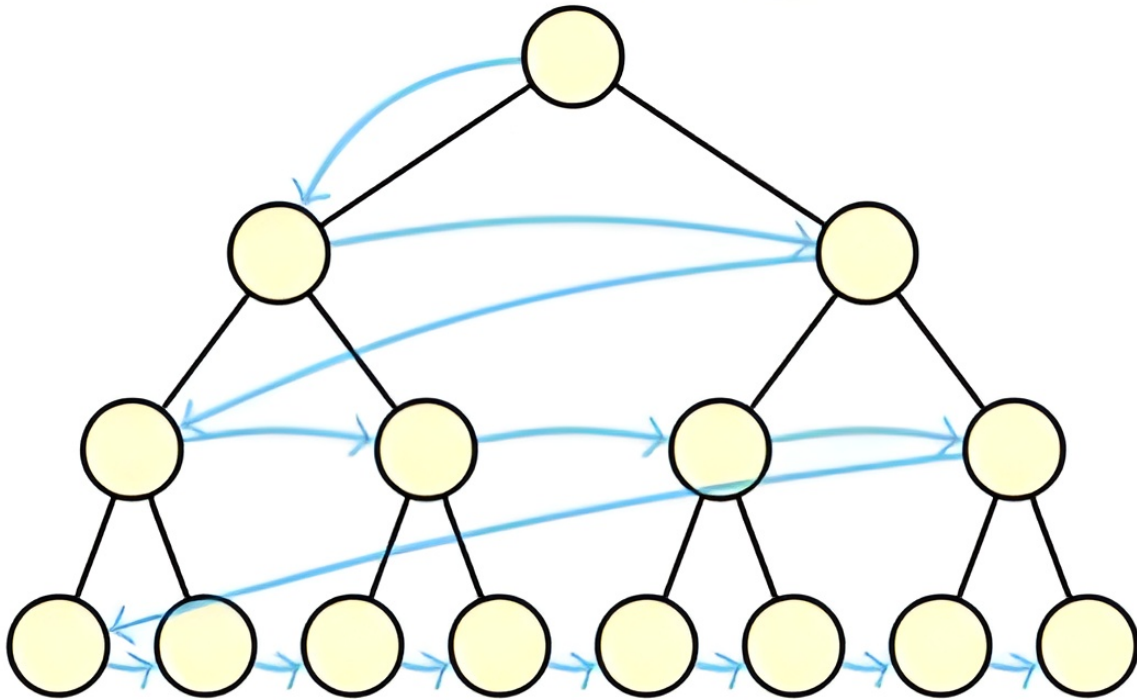
Давуу тал:

- Жингүй буюу бүх ирмэг ижил жинтэй гэж үзсэн граф дээр **хамгийн бага алхамтай** замыг баталгаатай олно.
- Гүйцэтгэлийн хувьд $O(|V| + |E|)$ хугацааны нарийвчлалтай.

Сул тал:

- Бүх төвшний оройг дараалал (**queue**) дотор хадгалах шаардлагатай тул санах ойн хэрэглээ их.
- Ирмэгийн жинг харгалзахгүй тул бодит замын **зай** эсвэл **хугацааны хувьд** заавал хамгийн оновчтой замыг олохгүй.

Breadth-First Search



Зураг 1: BFS diagram

2.2 DFS (Depth-First Search) алгоритм

DFS алгоритм нь графын оройнуудыг **аль болох гүн** чиглэлд дагаж хайдаг. Нэг салаа замыг төгсгөл хүртэл судалсны дараа буцаж (*backtracking*) өөр салаагаар үргэлжлүүлэн явдаг. Ингэснээр боломжит замуудын бүтцийг судлахад тохиромжтой.

Манай төсөлд DFS алгоритмыг:

- Хоёр цэгийн хооронд **нэг боломжит замыг** олох,
- BFS болон Dijkstra-тай харьцуулахад хэрхэн “төөрч” илүү урт зам сонгож болдгийг харуулахад ашигласан.

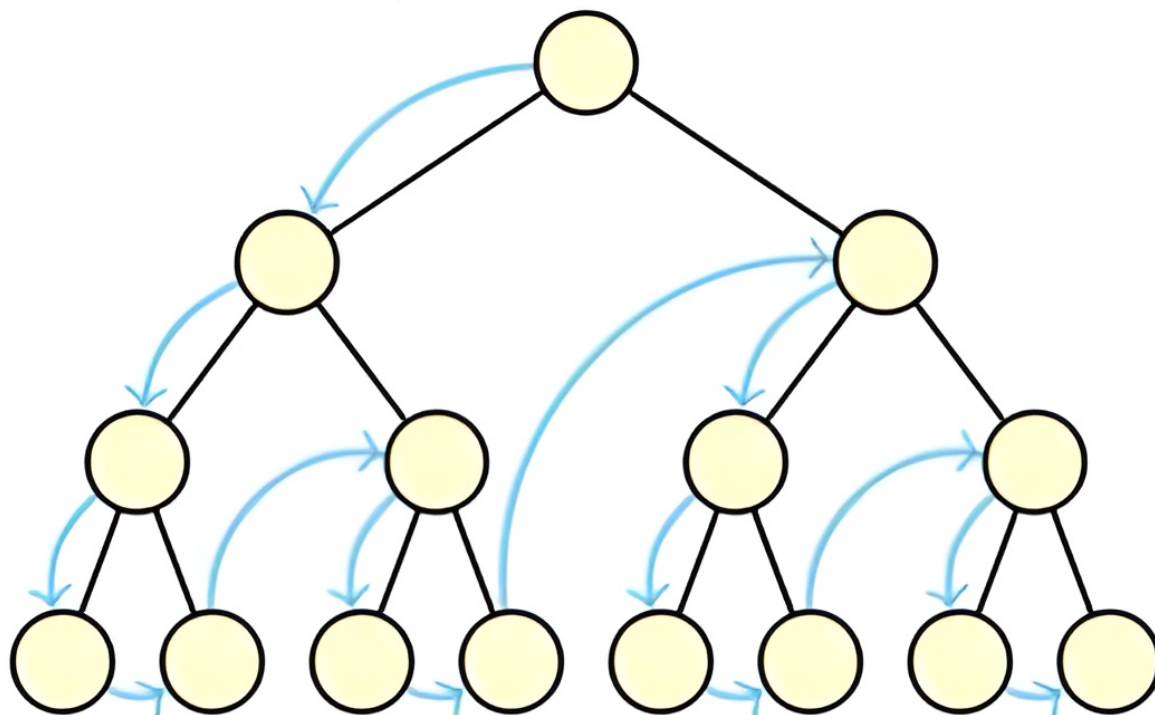
Давуу тал:

- Графын бүтцийг бүхэлд нь судлах, холбогдсон эсэхийг шалгах, цикл илрүүлэхэд тохиромжтой.
- Хэрэгжүүлэхэд хялбар, рекурс эсвэл стек (**stack**) ашиглан бичих боломжтой.

Сул тал:

- Замын уртыг харгалздаггүй учраас **хамгийн богино замыг баталгаатай олдоггүй**.
- Маш гүн граф дээр рекурсийн гүн ихсэж стек дүүрэх зэрэг эрстэлтэй.

Depth-First Search



Зураг 2: DFS diagram

2.3 Dijkstra алгоритм

Dijkstra алгоритм нь **жинтэй граф** дээр нэг эхлэх оройгоос бусад орой хүртэлх **хамгийн бага нийт жинтэй** замыг олох алгоритм юм. Манай төсөлд ирмэгийн жинг замын урт, хурдны хязгаар, замын төрлөөс (гол гудамж, туслах зам, шороон зам гэх мэт) хамааруулан тодорхойлсон.

Манай төсөлд Dijkstra алгоритмыг:

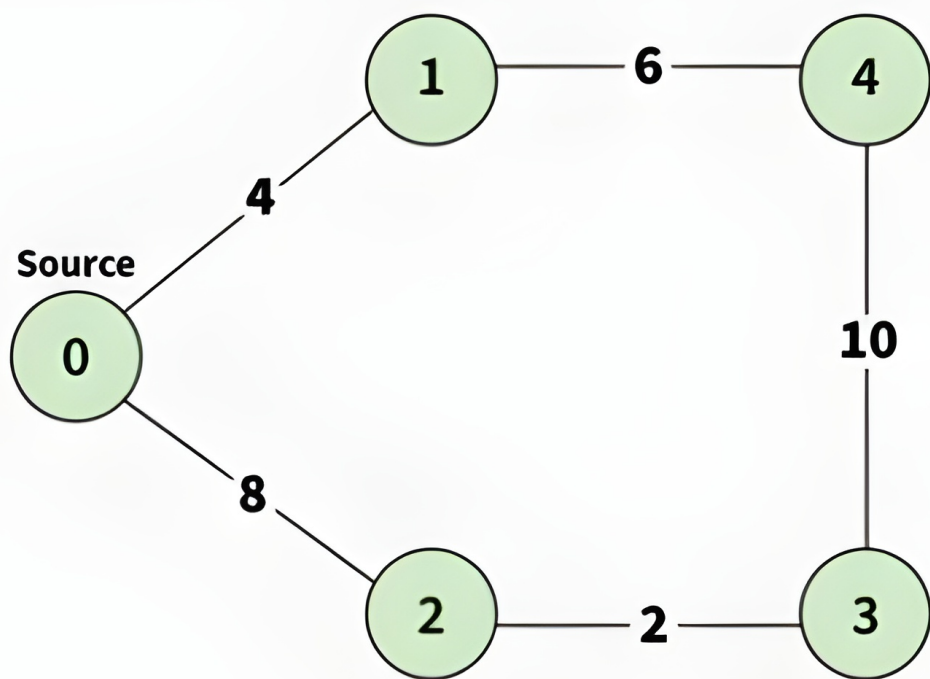
- Улаанбаатар хотын **бодит замын урт, нөхцөлийг тусгасан хамгийн оновчтой маршрутыг** тодорхойлоход,
- BFS/DFS-тай харьцуулахад **жинтэй граф дээр** ямар ялгаатай үр дүн гарч байгааг харуулахад ашигласан.

Давуу тал:

- Сөрөг жингүй граф дээр эхлэх цэгээс бусад цэг хүртэлх **хамгийн бага нийт жинтэй** замыг баталгаатай олно.
- Замын урт, явах хугацаа, зардал зэрэг бодит хэмжигдэхүүнүүдийг жин болгон ашиглах боломжтой.

Сул тал:

- Priority queue ашиглан олон дахин шинэчлэлт хийх тул BFS/DFS-аас илүү их **тооцоолол** шаарддаг.
- Сөрөг жинтэй ирмэгтэй граф дээр шууд ашиглаж болохгүй, буруу үр дүн өгөх эрстэлтэй.



Зыпар 3: Dijkstra diagram

3 ПРОГРАММЫН АЖИЛЛАГАА БА ХЭРЭГЖҮҮЛЭЛТ

3.1 Төслийн ерөнхий бүтэц

Төслийн программ нь **backend** болон **frontend** гэсэн хоёр үндсэн хэсгээс бүрдэнэ.

- **Backend (Python + Flask):**

- OpenStreetMap-ийн shapefile өгөгдлийг уншиж, замын сүлжээг граф болгон хувиргана.
- Граф дээр BFS, DFS, Dijkstra алгоритмуудаар зам хайж, үр дүнг JSON хэлбэрээр буцаана.

- **Frontend (HTML + JavaScript + Leaflet):**

- Улаанбаатар хотын газрын зургийг үзүүлж, хэрэглэгчээс эхлэх ба төгсгөлийн цэгийг сонгуулна.
- Алгоритмын сонголт хийх интерфейс үзүүлж, backend-аас ирсэн маршрутыг газрын зураг дээр polyline хэлбэрээр зурж харуулна.

Backend хэсгийн гол модуль файлууд нь `bfs.py`, `dfs.py`, `dijkstra.py`, `geo_utils.py`, `app.py` бөгөөд эдгээр нь тус бүр алгоритм, зай тооцоолол, веб серверийн логикийг агуулна.

3.2 Өгөгдөл боловсруулах шат

Газрын зургийн өгөгдөл нь OpenStreetMap-ийн shapefile (`.shp`, `.dbf`, `.shx`, `.prj`) форматтай байна. Эдгээрийг **GeoPandas** сан ашиглан уншиж дараах алхмуудаар граф болгон боловсруулсан.

1. Shapefile унших:

- `gpd.read_file()` функц ашиглан Улаанбаатар хотын замын шугаман геометрүүдийг (`LineString`) уншсан.
- `oneway`, `maxspeed`, `fclass`, `surface` зэрэг багануудыг замын чиглэл, хурдны хязгаар, замын төрлийг тодорхойлоход ашигласан.

2. Граф үүсгэх:

- Шугамын цэг бүрийг өргөрөг, уртраг бүхий `node` объект болгон хувиргаж,
- Дараалсан хоёр `node` хооронд `edge` үүсгэн, нэг чигийн эсвэл хоёр чигийн зам эсэхийг шалгаж графын ирмэгийг тодорхойлсон.

3. Haversine зай тооцох: Хоёр node хоорондын газар дээрх ойролцоо зайг дараах Haversine томъёогоор км нэгжээр тооцсон:

$$d = 2R \arcsin \sqrt{\sin^2 \left(\frac{\Delta\varphi}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)}$$

энд R бол дэлхийн радиус, φ нь өргөрөг, λ нь уртраг юм.

4. Хэрэглэгчийн цэгийг холбох:

- Хэрэглэгчийн сонгосон эхлэх ба төгсгөлийн координатыг тэмдэглэж,
- **хамгийн ойрын node**-ийг (nearest node) `geo_utils.py` дахь тусгай функцээр олж, граф дээрх эхлэл/төгсгөлийн орой болгон ашигласан.

3.3 Backend хэсгийн ажиллах зарчим

Backend нь **Flask** фреймворк дээр суурилсан бөгөөд REST API хэлбэрээр ажиллана. Гол endpoint нь:

- `/route?start=x_1,y_1&end=x_2,y_2&algo=bfs`

Энд:

- **start** — эхлэх цэгийн координат (**lon, lat**),
- **end** — төгсгөлийн цэгийн координат,
- **algo** — ашиглах алгоритм (**bfs, dfs, dijkstra**).

Endpoint-ийн дотоод урсгал:

1. Хэрэглэгчээс ирсэн координатыг задлан double төрөл уруу хөрвүүлнэ.
2. Эхлэх ба төгсгөлийн цэгийг граф дахь хамгийн ойрын node-той холбож, эхлэх/зорилтот оройг тодорхойлно.
3. Сонгосон алгоритмаас хамаарч:
 - **bfs.py** дахь BFS зам хайх функц,
 - **dfs.py** дахь DFS зам хайх функц,
 - **dijkstra.py** дахь Dijkstra зам хайх функцдуудагдана.
4. Олдсон маршрутын node-уудын координатын жагсаалт,
 - замын нийт урт,
 - ирмэгийн тоо,
 - алгоритмын гүйцэтгэлд зарцуулсан хугацаазэрэг үзүүлэлтийг тооцоолно.
5. Дээрх мэдээллийг JSON хэлбэрээр буцаана.

3.4 Frontend хэсгийн хэрэгжүүлэлт

Frontend хэсэг нь энгийн `index.html` файл дээр **Leaflet.js** сан ашиглан хэрэгжсэн.

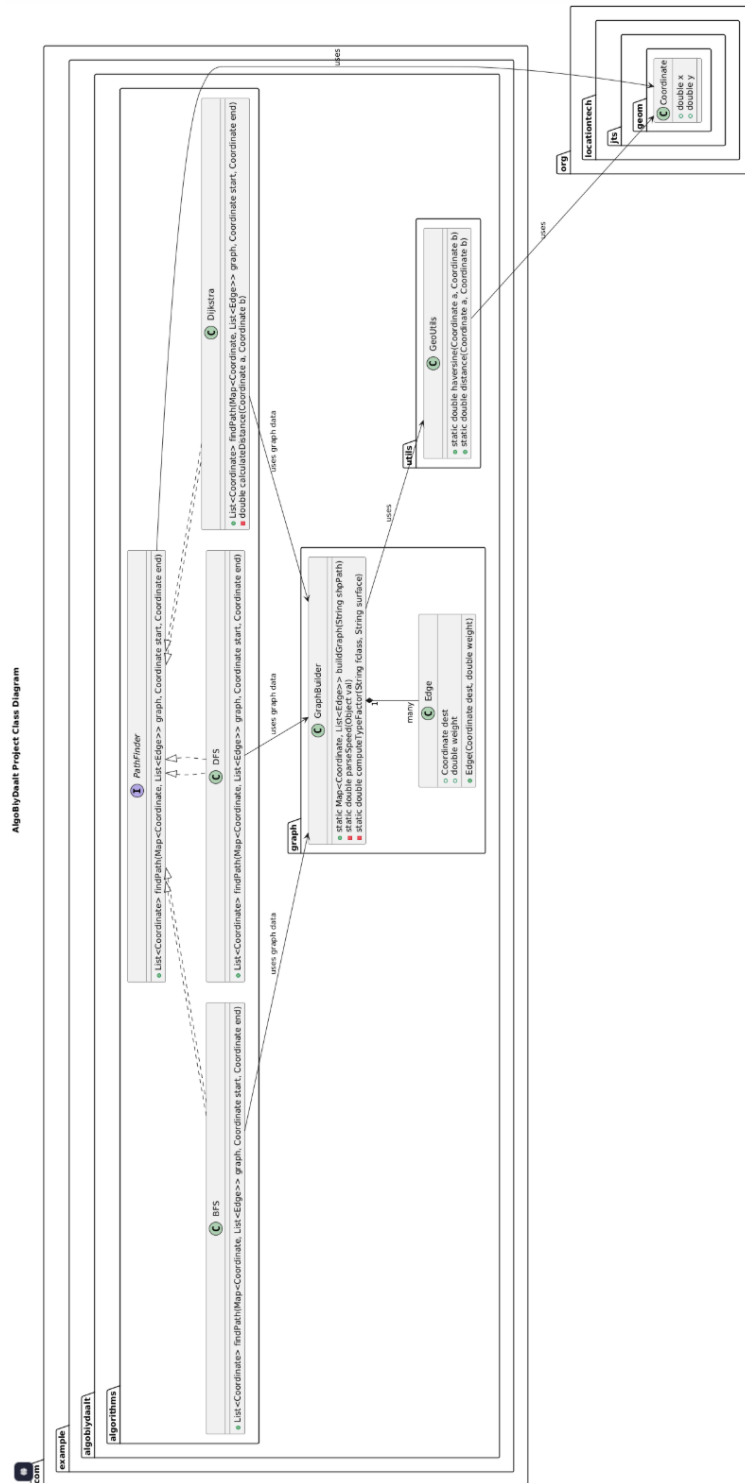
- **Газрын зураг дүрслэх:**
 - OSM-ийн tile server ашиглан Улаанбаатар хотын газрын зургийг Leaflet map дээр ачаална.
 - Хэрэглэгчид газрын зураг дээр чөлөөтэй **zoom, pan** хийх боломжтой.
- **Цэг сонгох:**
 - Газрын зураг дээр дарсан эхний цэгийг **эхлэх цэг**, хоёр дахь цэгийг **төгсгөлийн цэг** гэж үзэж, **marker** тавина.
 - Сонгосон координатуудыг формын талбаруудад эсвэл шууд JavaScript хувьсагчид хадгална.
- **Алгоритм сонголт:**
 - Dropdown (`<select>`) ашиглан BFS, DFS, Dijkstra алгоритмуудын аль нэгийг сонгох боломжтой.
 - “Зам тооцоолох” товчийг дармагц JavaScript-аас Flask backend уруу **fetch()** ашиглан `/route` уруу хүсэлт илгээнэ.

- **Маршрут дүрслэх:**

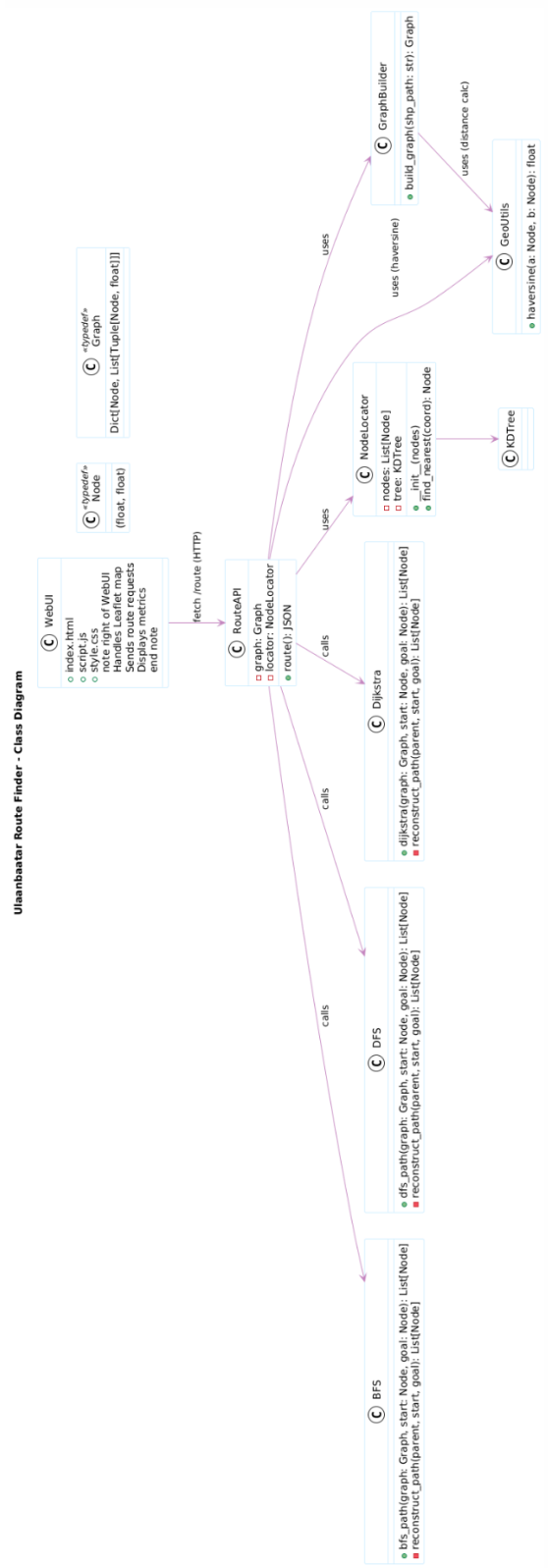
- Backend-аас ирсэн JSON доторх маршрутын координатын жагсаалтыг авч, Leaflet-ийн `L.polyline` ашиглан газрын зураг дээр **улаан шугам** болгон зурна.
- Эхлэх цэгийг ногоон, төгсгөлийн цэгийг улаан **marker**-аар тэмдэглэж, харагдах хүрээг автоматаар тохируулна.

Ингэснээр хэрэглэгч frontend интерфэйсээр дамжуулан Улаанбаатар хотын газрын зураг дээр хоёр цэг сонгож, өөр өөр алгоритмуудын үр дүнг бодитой харьцуулан харах боломжтой болж байна. ::contentReference[oaicite:0]index=0

4 Класс диаграмм



Зураг 4: Java класс диаграмм



Зураг 5: Python класс диаграмм

5 ХЭРЭГЛЭГЧИЙН ИНТЕРФЕЙС

5.1 Ерөнхий зохион байгуулалт

Төслийн хэрэглэгчийн интерфейс нь **web** суурьтай нэг хуудсан (*single-page*) аппликейшн бөгөөд HTML, CSS, JavaScript болон Leaflet.js ашиглан хэрэгжсэн. Интерфейс нь голчлон дараах гурван бүсээс бүрдэнэ.

- **Газрын зураг (map area)**
Улаанбаатар хотын OpenStreetMap суурь газрын зургийг харуулна. Хэрэглэгч газрын зураг дээр **zoom**, **pan** хийх боломжтой.
- **Удирдлагын самбар (control panel)**
Эхлэх ба төгсгөлийн цэг сонгох, алгоритм сонгох (BFS, DFS, Dijkstra), зам тооцоолох болон дахин тохируулах товчуудаас бүрдэнэ.
- **Үр дүнгийн хэсэг (result panel)**
Олдсон маршрутын талаарх мэдээлэл — замын урт, алхмын тоо, алгоритмын гүйцэтгэлийн хугацаа зэрэг үзүүлэлтийг текст хэлбэрээр харуулна.

Эдгээр хэсгүүд нь нэг хуудсанд зэрэгцэн байрлаж, хэрэглэгчийн оролтын дагуу газрын зураг болон мэдээллийн хэсэг бодит цагт шинэчлэгддэг.

5.2 Газрын зураг дээрх харилцан үйлдэл

Газрын зураг дээрх гол харилцан үйлдэл нь эхлэх болон төгсгөлийн цэгийг сонгох, маршрутыг харах явдал юм.

- **Эхлэх цэг сонгох:**
 - Хэрэглэгч газрын зураг дээр анхны удаа дарсан цэгийг **эхлэх цэг** гэж үзнэ.
 - Тухайн цэг дээр **ногоон** өнгийн **marker** байрлуулж, координатыг дотоод хувьсагчид хадгална.
- **Төгсгөлийн цэг сонгох:**
 - Хоёр дахь удаа дарсан цэгийг **төгсгөлийн цэг** гэж тэмдэглэнэ.
 - Улаан өнгийн **marker** ашиглан газрын зураг дээр харагдуулна.
- **Цэгүүдийг өөрчлөх:**
 - Хэрэв хэрэглэгч цэгээ буруу сонгосон бол дахин дарахад **marker**-үүд шинэчлэгдэнэ.
 - Мөн “Reset” товч дарснаар бүх **marker** болон өмнөх маршрут цэвэрлэгдэж, шинээр сонголт хийх боломжтой.

5.3 Алгоритм сонголт ба зам тооцоолол

Интерфейс дээр алгоритм сонгох **dropdown** хэсэг байрлана. Хэрэглэгч дараах сонголтуудаас нэгийг нь сонгоно.

- **BFS** — хамгийн цөөн алхамтай (ирмэгийн тоо бага) замыг олох.
- **DFS** — боломжит нэг замыг гүн чиглэлээр хайх.
- **Dijkstra** — жингийн хувьд хамгийн бага нийт зардалтай (замын урт, хугацаа) замыг олох.

Хэрэглэгч алгоритм сонгосны дараа:

- **“Зам тооцоолох”** товчийг дарснаар frontend нь backend уруу HTTP GET хүсэлт илгээнэ.
- Хүсэлтийн параметрууд:
 - **start = x_1, y_1** — эхлэх цэгийн координат,

- `end = x_2, y_2` — төгсгөлийн цэгийн координат,
- `algo = bfs/dfs/dijkstra` — сонгосон алгоритм.
- Backend нь тухайн алгоритмаар замыг тооцоолж, координатын жагсаалт болон гүйцэтгэлийн мэдээллийг JSON хэлбэрээр буцаана.

5.4 Маршрутын дүрслэл ба үр дүнгийн мэдээлэл

Backend-аас амжилттай хариу ирсэн тохиолдолд:

- **Маршрут дүрслэх:**
 - JSON-д ирсэн координатуудыг Leaflet-ийн `L.polyline` функцээр `polyline` болгон үүсгэж, газрын зураг дээр **улаан шугам** хэлбэрээр харуулна.
 - Эхлэх ба төгсгөлийн **marker**-уудын хамтаар харагдахуйц байдлаар газрын зураг автоматаар `fitBounds` хийж тухайн маршрутыг төвлөрүүлнэ.
- **Текст үр дүн:**
 - Сонгосон алгоритмын нэр (`bfs`, `dfs`, `dijkstra`),
 - Замын нийт урт (Haversine зайгаар, км нэгжээр),
 - Алхмын тоо (дамжсан ирмэгийн тоо),
 - Алгоритмын гүйцэтгэлийн хугацаа (секундээр) зэрэг үзүүлэлтийг мэдээллийн хэсэгт харуулна.

Хэрэв маршрут олддоогүй тохиолдолд `polyline` зурахгүй бөгөөд үр дүнгийн хэсэгт “Зам олдсонгүй” гэсэн анхааруулга гарна.

5.5 Алдаа болон зааварчилгын мессежүүд

Хэрэглэгчийн алдаатай оролт, серверийн алдаа гарсан үед интерфейс дараах байдлаар мэдээлэл өгнө.

- **Эхлэх эсвэл төгсгөлийн цэг сонгоогүй үед:**
“Эхлэх болон төгсгөлийн цэгийг газрын зураг дээр сонгоно уу” гэсэн анхааруулга текстээр гарна.
- **Алгоритм сонгоогүй үед:**
Алгоритмын `dropdown`-аас сонголт хийгээгүй бол анхдагч байдлаар BFS ашиглах эсвэл алгоритм сонгохыг сануулах мессеж гаргахаар зохион байгуулсан.
- **Серверийн алдаа:**
Backend-аас **error** талбартай JSON хариу ирсэн тохиолдолд тухайн алдааны мессежийг интерфейс дээр текстээр харуулна (жишээ нь: “Өгөгдлийг уншихад алдаа гарлаа”, “Зам олох явцад алдаа гарлаа” гэх мэт).

5.6 Хэрэглэгчийн үйлдлийн ерөнхий дараалал

Хэрэглэгч аппликейшныг ашиглах стандарт урсгал нь дараах байдалтай.

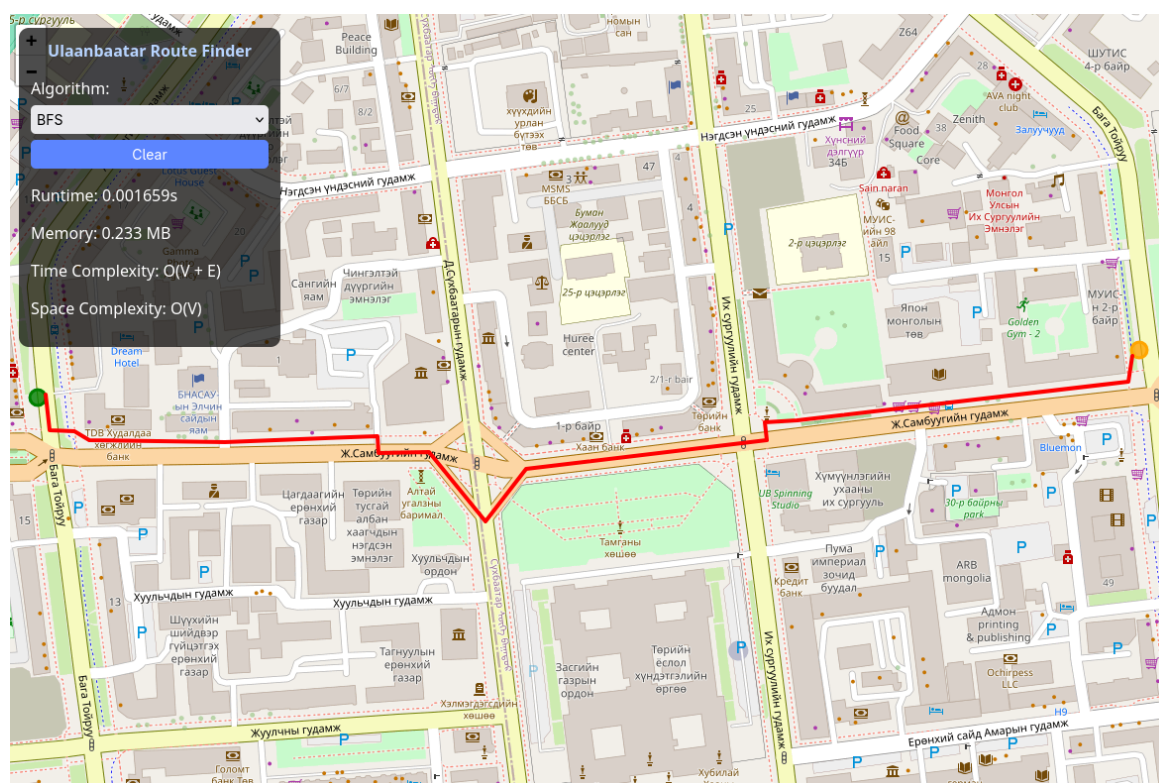
1. Газрын зураг дээр эхлэх цэгээ дарж сонгоно (ногоон `marker` гарна).
2. Газрын зураг дээр төгсгөлийн цэгээ дарж сонгоно (улаан `marker` гарна).
3. Алгоритм сонгох `dropdown`-аас BFS, DFS эсвэл Dijkstra-гийн аль нэгийг сонгоно.
4. “Зам тооцоолох” товчийг дарж, backend серверээр дамжуулан замыг тооцоолуулна.

5. Газрын зураг дээр гарч ирсэн маршрутыг, мөн үр дүнгийн хэсэгт үзүүлсэн замын урт, алх-мын тоо, гүйцэтгэлийн хугацаа зэрэг үзүүлэлтийг харьцуулж үзнэ.
6. Хэрэв өөр маршрут туршихыг хүсвэл “Reset” товчийг дарж өмнөх мэдээллийг цэвэрлээд, шинэ цэгүүд сонгож дээрх алхмуудыг давтана.

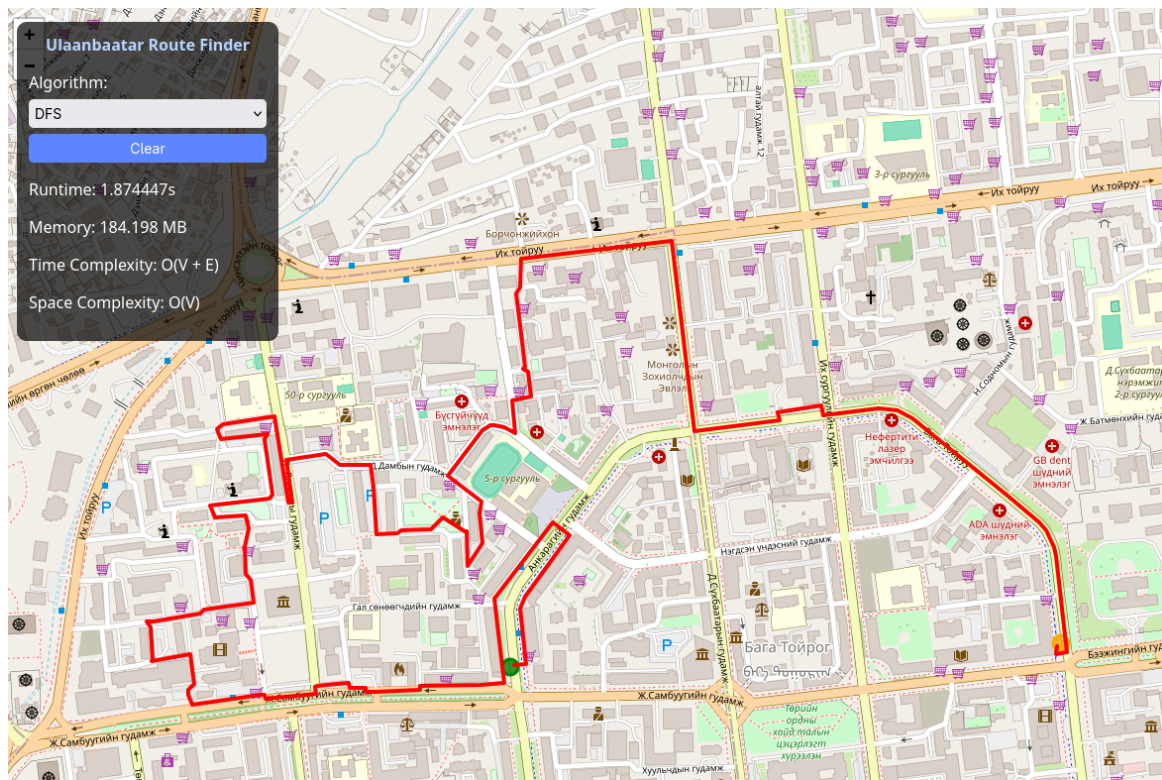
Ингэснээр хэрэглэгч графын хайлтын гурван өөр алгоритмыг **адил орчинд, нэг интерфейсээр** дамжуулан харьцуулж, тэдгээрийн ажиллах зарчмыг бодит жишээн дээр ойлгох боломжтой болно.

5.7 Алгоритмуудын дүрслэл

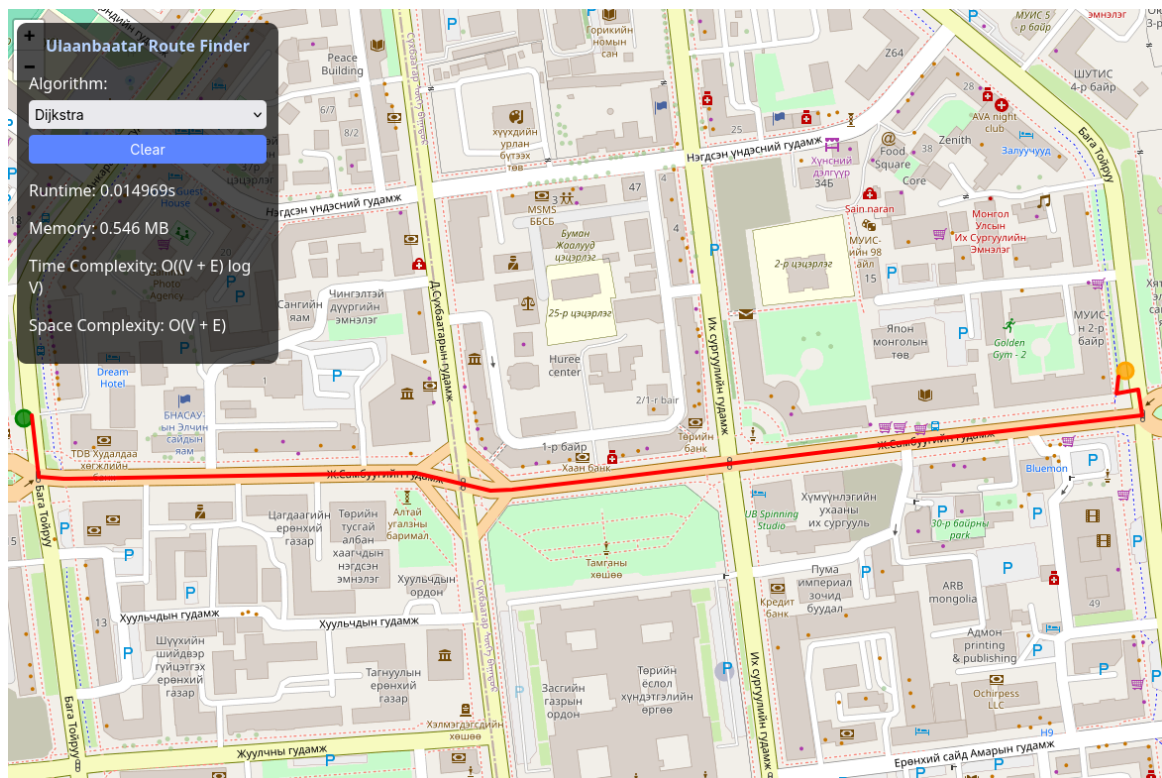
Доорх зургууд нь гурван төрлийн алгоритмын (BFS, DFS, Dijkstra) зам олох үйл явцыг жишээ байдлаар харуулсан болно. Эдгээр зургуудыг дараа нь бодит үр дүнгийн зургаар орлуулна.



Зураг 6: BFS алгоритмын зам олох жишээ зураг



Зураг 7: DFS алгоритмын зам олох жишээ зураг



Зураг 8: Dijkstra алгоритмын зам олох жишээ зураг

6 ТЕСТ БА БАТАЛГААЖУУЛАЛТ

6.1 Зорилго

Энэхүү хэсгийн зорилго нь маршрут тооцоолох алгоритмууд (BFS, DFS, Dijkstra) болон нийт системийн ажиллагааг **үнэн зөв, логикийн хувьд нийцтэй** байгааг шалгаж, баталгаажуулах явдал юм.

Тестүүд нь дараах үндсэн асуултуудад хариу өгөхийг зорилоо.

- Алгоритмууд онолынхоо дагуу зөв зам олж чадаж байна уу?
- BFS нь ирмэгийн тоогоор **хамгийн цөөн алхамтай** замыг олж байна уу?
- Dijkstra нь жингийн хувьд **хамгийн бага нийт зардалтай** замыг баталгаатай олж байна уу?
- DFS нь хүрч болох зорилтот цэг уруу **зам олж чаддаг** (reachability) эсэх?
- Frontend–backend интеграцын үед буруу оролт, серверийн алдаа гарахад систем зөв мессеж үзүүлж байна уу?

6.2 Инвариант дээр суурилсан баталгаажуулалт

Алгоритм бүрд онолын төвшний **инвариант** буюу үргэлж биелж байх ёстой шинж чанарыг кодын тестээр шалгасан.

BFS — хамгийн бага алхамтай зам

BFS-ийн хувьд дараах шаардлага тавигдана.

- Гаралтын зам дээрх ирмэгийн тоо нь тухайн хоёр орой хоорондох **хамгийн бага боломжит hop**-ийн тоотой тэнцүү байх ёстой.
- Үүнийг жижиг графууд дээр гаралтын замын уртыг бүх боломжит замтай гар аргаар харьцуулж шалгасан.

DFS — хүрч болох эсэх (reachability)

DFS-ийн хувьд:

- Хэрэв зорилтот орой эхлэх оройгоос хүрч болох хэсэгт байвал, DFS нь **ямар нэг зам** заавал олж буцаах ёстой.
- Хэрэв хүрч болохгүй бол DFS-ийн буцаасан замын жагсаалт хоосон байх ёстой.

Энэ зорилгоор:

- Холбогдсон граф дээр DFS үргэлж хоосон бус зам буцааж байгааг,
- Зорилтот орой холбогдоогүй тусдаа бүрэлдэхүүнд байх үед хоосон зам буцааж байгааг шалгасан.

Dijkstra — жингийн хувьд хамгийн бага зам

Dijkstra алгоритмын хувьд:

- Олдсон замын нийт жин нь бусад боломжит замуудаас **их биш** байх ёстой.

Жижиг жинтэй графууд дээр:

- Бүх боломжит замын жинг гар аргаар (эсвэл тусдаа скриптээр) бодож,
- Dijkstra-ийн гаргасан шийдлийн жинг тэдгээртэй харьцуулж,

- ямар ч тохиолдолд Dijkstra-ийн сонгосон замын жин **хамгийн бага** байгаа эсэхийг шалгасан.

6.3 Интеграцын болон гар тестүүд

Алгоритмууд зөв ажиллаж байгааг нэгж төвшинд баталгаажуулсны дараа, frontend-backend интеграцын тестийг **газрын зурагтай орчинд гар тест** хэлбэрээр хийсэн.

1. Хэвийн хэрэглээний тохиолдлууд

- Улаанбаатар хотын төв хэсгээс өөр нэг байршил уруу (жишээ нь Сүхбаатарын талбай → Их дэлгүүр) BFS, DFS, Dijkstra алгоритмуудаар маршрут тооцоолсон.
- Бүх алгоритм:
 - Маршрут олж, газрын зураг дээр polyline хэлбэрээр буцааж байгааг,
 - Эхлэх, төгсгөлийн marker зөв байрлаж байгааг,
 - Замын урт, алхмын тоо, хугацааны утгууд бодитой (0-аас их, хоорондоо утгын хувьд логиктой) байгааг
 шалгасан.

2. Буруу оролт ба алдааны мессеж

Дараах тохиолдлуудад алдааны менежментийг шалгасан.

- Эхлэх болон төгсгөлийн цэгийг сонгоогүй үед:
 - “Эхлэх болон төгсгөлийн цэгийг газрын зураг дээр сонгоно уу” гэх утгатай анхааруулга гарч байгаа эсэх.
- Алгоритм сонгоогүй / буруу параметр илгээсэн үед:
 - Backend “Invalid algorithm” гэсэн алдааг буцааж, frontend талд алдааны мессеж хэвлэж байгааг шалгасан.
- Серверийн дотоод алдаа (жишээ нь sharefile олдоогүй, граф ачаалагдаагүй гэх мэт) үүсгэхэд:
 - HTTP 500 алдаа ирж, хэрэглэгчид ойлгомжтой текст мессежээр алдаа гарсныг мэдэгдэж байгааг шалгасан.

6.4 Тестийн үр дүнгийн дүгнэлт

Дээрх нэгж тестүүд, инвариант дээр суурилсан шалгалтууд болон интеграцын гар тестүүдийн үр дүнд:

- BFS нь ирмэгийн тоогоор хамгийн цөөн алхамтай замыг,
- DFS нь хүрч болох зорилтот цэг уруу зам олж чаддаг болохыг,
- Dijkstra нь жингийн хувьд хамгийн бага нийт зардалтай замыг олдог болохыг онолын болон практик төвшинд баталгаажуулсан.

Ингэснээр программ нь зөвхөн онолын хувьд төдийгүй, бодит өгөгдөл дээр ч найдвартай ажиллаж буйг харлаа.

6.5 Туршилтын үр дүнгийн хүснэгт (Python үр дүн)

Хүснэгт 1: Алгоритмуудын туршилтын гүйцэтгэлийн үр дүн (Python)

Алгоритм	Туршилт №	Хугацаа (ms)	Санах ой (MB)	Дунд. хугацаа (ms)	Дунд. санах ой (MB)
BFS	1	106.76	3.82	191.21	6.13
	2	101.81	3.82		
	3	385.85	11.51		
	4	140.49	5.76		
	5	221.15	5.76		
DFS	1	195.17	5.77	1469.33	38.84
	2	8.06	0.34		
	3	99.99	3.82		
	4	3544.60	92.13		
	5	3498.81	92.12		
Dijkstra	1	100.00	1.50	210.44	4.88
	2	236.18	5.65		
	3	230.85	5.66		
	4	97.81	2.22		
	5	387.36	9.38		

6.6 Туршилтын үр дүнгийн хүснэгт (Java үр дүн)

Хүснэгт 2: Алгоритмуудын туршилтын гүйцэтгэлийн үр дүн (Java)

Алгоритм	Туршилт №	Замын урт	Хугацаа (ms)	Дунд. хугацаа (ms)	Дунд. замын урт
BFS	1	59	13.70	20.60	84.8
	2	65	26.00		
	3	85	21.30		
	4	101	29.60		
	5	114	12.40		
DFS	1	40697	2263.90	947.48	28297.2
	2	3897	8.40		
	3	40778	2183.60		
	4	3560	5.70		
	5	52554	275.80		
Dijkstra	1	59	3.20	14.22	88.6
	2	140	13.10		
	3	65	1.70		
	4	113	51.00		
	5	66	2.10		

6.7 Алгоритмуудын дундаж гүйцэтгэлийн харьцуулалт (Python vs Java)

Хүснэгт 3: Python болон Java хэрэгжилтийн дундаж гүйцэтгэлийн харьцуулалт

Алгоритм	Python дунд. хугацаа (ms)	Python дунд. санах ой (MB)	Java дунд. хугацаа (ms)	Java дунд. замын урт
BFS	191.21	6.13	20.60	84.8
DFS	1469.33	38.84	947.48	28297.2
Dijkstra	210.44	4.88	14.22	88.6

7 ДҮГНЭЛТ

7.1 Ерөнхий дүгнэлт

Энэхүү бие даалтын ажлаар графын хайлтын үндсэн алгоритмууд болох **BFS**, **DFS**, **Dijkstra**-гийн онолын ойлголтыг бодит өгөгдөл дээр хэрэгжүүлэн шалгалаа. OpenStreetMap-ын shapefile өгөгдлөөс Улаанбаатар хотын замын сүлжээ үүсгэж, түүнийг графын *node* болон *edge* бүтэцтэй болгосноор:

- Хэрэглэгчийн сонгосон хоёр цэгийн хооронд автомашинаар зорчих боломжит маршрут гаргах,
- Алгоритм бүрээр тооцоологдсон замын урт, алхмын тоо, жингийн ялгааг харьцуулах,
- Графын алгоритмуудыг веб суурьтай, интерактив орчинд үзүүлэх боломжтой системийг амжилттай боловсруулж, ажиллуулсан.

7.2 Алгоритмуудын харьцуулалтын дүгнэлт

Төслийн хэрэгжилт, тестийн үр дүнгээс дараах гол дүгнэлтийг хийж болно.

- **BFS** алгоритм нь ирмэгийн жин харгалзахгүй орчинд **хамгийн цөөн алхамтай** (hop багатай) замыг баталгаатай олдог бөгөөд богино алхамтай маршрут хэрэгтэй тохиолдолд тохиромжтой.
- **DFS** алгоритм нь бүхэл графын бүтцийг судлах, хүрч болох эсэхийг шалгах зэрэгт тохиромжтой ч **замын уртыг оптимизацлахад** зориулагдаагүйг практик туршилтаар дахин батлав.
- **Dijkstra** алгоритм нь жинтэй граф дээр **жин/уртын хувьд хамгийн оновчтой** маршрутыг олж чаддаг бөгөөд бодит замын урт, явах хугацаа зэргийг тооцох шаардлагатай **navigation** төрлийн системд илүү тохиромжтой нь харагдсан.

Ингэснээр онолын төвшинд үзсэн дүгнэлтүүдийг, Улаанбаатар хотын замын бодит өгөгдөл дээр туршиж, практик байдлаар баталгаажуулсан.

7.3 Программын хэрэгжилтийн ололт амжилт

Хэрэгжүүлсэн программын хувьд дараах ололт, давуу талууд ажиглагдлаа.

- OpenStreetMap-ийн shapefile өгөгдлөөс граф үүсгэх pipeline-г боловсруулж, Naversine томьёогоор node хоорондын зайг тооцон, жингүүдийг тодорхойлсон.
- Python + Flask ашиглан **REST API** хэлбэрийн backend сервер бичиж, алгоритмуудыг вебээр дуудах боломжтой болгосон.
- HTML + JavaScript + Leaflet.js ашиглан **газрын зураг дээр интерактив интерфэйс** үүсгэж, хэрэглэгчийн сонгосон цэгүүдийн хоорондын маршрутыг шууд үзүүлэх боломжийг бүрдүүлсэн.
- BFS, DFS, Dijkstra алгоритмуудыг тус тусад нь модульчлон хэрэгжүүлж, кодын төвшинд дахин ашиглах, өргөтгөхөд тохиромжтой бүтэцтэй болгосон.

Эдгээрийн үр дүнд графын алгоритмыг зөвхөн консоль дээр бус, газрын зурагтай, хэрэглэгчдэд ойлгомжтой орчинд харуулах боломжтой болсон.

7.4 Хязгаарлалт ба цаашдын сайжруулалт

Төслийн хүрээнд тодорхой хэмжээний хязгаарлалт, сайжруулах боломжууд ажиглагдлаа.

- **Өгөгдлийн чанар:** OpenStreetMap-ийн замын өгөгдөл зарим бүсэд дутуу, ангилал нь төгс биш байж болдог тул маршрутын үр дүнг 100% бодит байдлын дагуу гэж үзэх боломжгүй.
- **Гүйцэтгэл:** Одоогийн хэрэгжилт нь нэг сервер дээр ажиллаж байгаа тул маш том граф дээр (бүх улс, олон хот зэрэг) ажиллуулахад гүйцэтгэлийн оновчлол шаардагдана.
- **Нэмэлт хүчин зүйлс:** Замын түгжрэл, нэг чиглэлт гудамж, гэрлэн дохио, хурдны хязгаарын бодит цагийн мэдээлэл гэх мэт хүчин зүйлсийг алгоритмд оруулаагүй.

Цаашид дараах хөгжүүлэлтүүдийг хийх боломжтой гэж үзэв.

- A^* зэрэг илүү оновчтой хайлтын алгоритмуудыг нэмэх.
- Замын түгжрэл, бодит цагийн урсгалын мэдээлэлтэй холбон **динамик маршрут** тооцдог болгох.
- Хэрэглэгчийн интерфэйсийг сайжруулж, олон цэгийн маршрутууд (waypoint), өөр өөр зорилго (жишээ нь алхах, дугуй, нийтийн тээвэр) нэмэх.

7.5 Хувийн суралцах үр дүн

Энэхүү бие даалт нь зөвхөн алгоритмын онолыг давтаж бататгахаас гадна дараах ур чадварыг нэмэгдүүлэхэд чухал нөлөөтэй байлаа.

- Графын өгөгдлийг бодит газрын зургийн өгөгдлөөс үүсгэх, түүнийг алгоритмд тохирох бүтэц уруу хөрвүүлэх.
- Backend–frontend архитектур зохиож, REST API ашиглан сервер, клиент хоёрын хооронд өгөгдөл солилцох.
- Алгоритм бүрийн давуу ба сул талыг туршилтаар харьцуулж, ямар нөхцөлд аль алгоритм тохиромжтойг дүгнэх.

Ингэснээр графын алгоритмуудын онолын болон практик хэрэглээг илүү гүнзгий ойлгож, бодит амьдралын асуудалд хэрхэн ашиглаж болох талаар туршлага хуримтлуулсан болно.

Ашигласан ном

- [1] Thomas H. Cormen and others. *Introduction to Algorithms*. 3 edition. Cambridge, MA: MIT Press, 2009.
- [2] GeoPandas developers. *GeoPandas Documentation*. Accessed 2025-11-14. 2025. URL: <https://geopandas.org/>.
- [3] OpenStreetMap contributors. *OpenStreetMap Wiki*. Accessed 2025-11-14. OpenStreetMap Foundation. 2025. URL: <https://wiki.openstreetmap.org/>.
- [4] Wikipedia contributors. *Breadth-first search*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Breadth-first_search.
- [5] Wikipedia contributors. *Depth-first search*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Depth-first_search.
- [6] Wikipedia contributors. *Dijkstra's algorithm*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.

Ашигласан ном

- [1] Thomas H. Cormen **and others**. *Introduction to Algorithms*. 3 **edition**. Cambridge, MA: MIT Press, 2009.
- [2] GeoPandas developers. *GeoPandas Documentation*. Accessed 2025-11-14. 2025. URL: <https://geopandas.org/>.
- [3] OpenStreetMap contributors. *OpenStreetMap Wiki*. Accessed 2025-11-14. OpenStreetMap Foundation. 2025. URL: <https://wiki.openstreetmap.org/>.
- [4] Wikipedia contributors. *Breadth-first search*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Breadth-first_search.
- [5] Wikipedia contributors. *Depth-first search*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Depth-first_search.
- [6] Wikipedia contributors. *Dijkstra's algorithm*. Accessed 2025-11-14. Wikipedia, The Free Encyclopedia. 2025. URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.